

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
# Load Titanic dataset
df = sns.load_dataset("titanic")

# Display the first 5 rows
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

```
# Check the dataset information
print("Shape of dataset:", df.shape)
```

```
# Check column names
print("\nColumns:")
print(df.columns.tolist())
```

```
# Check missing values
print("\nMissing values:")
print(df.isnull().sum())
```

Shape of dataset: (891, 15)

Columns:
['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',

Missing values:

```
survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
dtype: int64
```

```
# Select the features and target
features = ["pclass", "sex", "age", "sibsp", "parch", "fare", "embarked"]
```

```
X = df[features]
y = df["survived"]
```

```
# Display the selected data
X.head()
```

	pclass	sex	age	sibsp	parch	fare	embarked
0	3	male	22.0	1	0	7.2500	S
1	1	female	38.0	1	0	71.2833	C
2	3	female	26.0	0	0	7.9250	S
3	1	female	35.0	1	0	53.1000	S
4	3	male	35.0	0	0	8.0500	S

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Training data: (712, 7)
Testing data: (179, 7)

```
# Encode categorical features 'sex' and 'embarked'
le = LabelEncoder()
X_train['sex'] = le.fit_transform(X_train['sex'])
X_test['sex'] = le.transform(X_test['sex'])

X_train['embarked'] = le.fit_transform(X_train['embarked'])
X_test['embarked'] = le.transform(X_test['embarked'])

print("X_train after encoding:")
display(X_train.head())
print("\nX_test after encoding:")
display(X_test.head())
```

X_train after encoding:

	pclass	sex	age	sibsp	parch	fare	embarked
331	1	1	45.5	0	0	28.5000	2
733	2	1	23.0	0	0	13.0000	2
382	3	1	32.0	0	0	7.9250	2
704	3	1	26.0	1	0	7.8542	2
813	3	0	6.0	4	2	31.2750	2

X_test after encoding:

	pclass	sex	age	sibsp	parch	fare	embarked
709	3	1	30.505845	1	1	15.2458	0
439	2	1	31.000000	0	0	10.5000	2
840	3	1	20.000000	0	0	7.9250	2
720	2	0	6.000000	0	1	33.0000	2
39	3	0	14.000000	1	0	11.2417	0

```
# Handle missing values in 'age' column with mean imputation
X_train['age'].fillna(X_train['age'].mean(), inplace=True)
X_test['age'].fillna(X_test['age'].mean(), inplace=True)

# Handle missing values in 'embarked' column with mode imputation (most frequent value)
X_train['embarked'].fillna(X_train['embarked'].mode()[0], inplace=True)
X_test['embarked'].fillna(X_test['embarked'].mode()[0], inplace=True)

print("Missing values in X_train after imputation:")
print(X_train.isnull().sum())
print("\nMissing values in X_test after imputation:")
print(X_test.isnull().sum())
```

Missing values in X_train after imputation:

pclass	0
sex	0
age	0

```
sibsp      0
parch      0
fare      0
embarked   0
dtype: int64
```

Missing values in X_test after imputation:

```
pclass     0
sex        0
age        0
sibsp      0
parch      0
fare       0
embarked   0
dtype: int64
```

/tmp/ipykernel_608/1559926657.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
X_train['age'].fillna(X_train['age'].mean(), inplace=True)
```

/tmp/ipykernel_608/1559926657.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
X_test['age'].fillna(X_test['age'].mean(), inplace=True)
```

/tmp/ipykernel_608/1559926657.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
X_train['embarked'].fillna(X_train['embarked'].mode()[0], inplace=True)
```

/tmp/ipykernel_608/1559926657.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
X_test['embarked'].fillna(X_test['embarked'].mode()[0], inplace=True)
```

```
# Create the Random Forest model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

print("Model training completed successfully!")
```

Model training completed successfully!

```
# Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Accuracy percentage:", accuracy * 100, "%")
```

Accuracy: 0.8156424581005587
Accuracy percentage: 81.56424581005587 %

```
# Display the classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

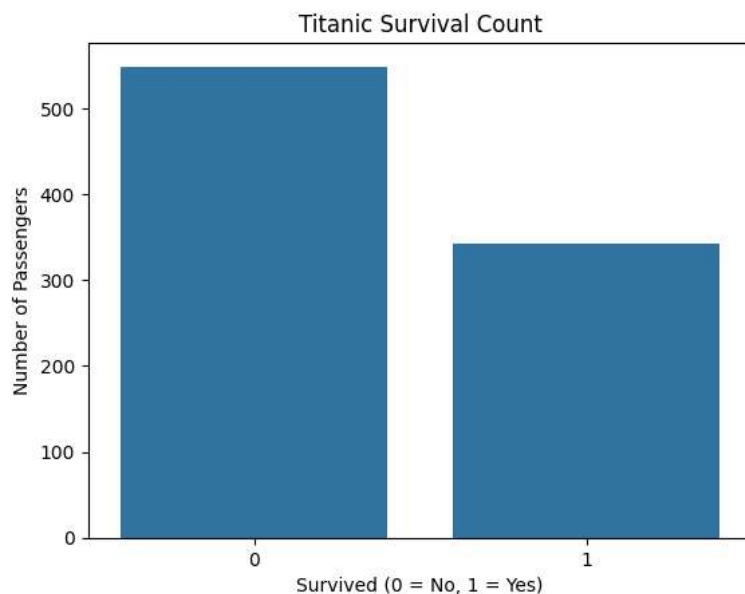
0               0.83        0.87        0.85         105
1               0.80        0.74        0.77          74
```

accuracy			0.82	179
macro avg	0.81	0.80	0.81	179
weighted avg	0.81	0.82	0.81	179

```
# Visualize actual survival counts
sns.countplot(x="survived", data=df)

plt.title("Titanic Survival Count")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Number of Passengers")

plt.show()
```



```
# Show actual vs predicted survival
results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

print(results.head(10))
```

	Actual	Predicted
0	1	0
1	0	0
2	0	0
3	1	1
4	1	0
5	1	1
6	1	1
7	0	0
8	1	1
9	1	1